

Universidad del Valle de Guatemala

Facultad de Ingeniería · Departamento de Ciencias de la Computación

CC3067 Redes · Sección 10

Laboratorio 5

Mini datacenter virtualizado

WordPress en arquitectura de 2 capas con HTTPS, SFTP y administración remota

Integrante	Carné	Responsabilidad principal
Nicolás Concuá	23197	Capa web, WordPress, permisos, SFTP y HTTPS
Daniela Ramírez	23053	Infraestructura VirtualBox, red, acceso y demostración
Fernando Mendoza	19644	SSH, firewall UFW, base de datos y pruebas

Guatemala, 10 de noviembre de 2026

Repositorio: github.com/nicoCT04/Lab5-Redes

Resumen

Se implementó un sitio WordPress separado en dos capas sobre dos máquinas virtuales con Ubuntu Server 26.04.1 LTS en VirtualBox 7.2: `lab5-web` (Nginx, PHP-FPM y WordPress) y `lab5-db` (MySQL 8.4 LTS). Ambas se comunican por una red host-only privada (192.168.56.0/24) y se administran exclusivamente por SSH desde el equipo host, incluida la gestión de la base de datos. El sitio se publica solo por HTTPS con un certificado de Let's Encrypt válido para el navegador y se accede por el nombre `redes-lab5-wp.duckdns.org`, que el host resuelve localmente mediante su archivo hosts.

Las decisiones de diseño siguen un criterio de mínimo privilegio y defensa en profundidad: servicios en puertos no conocidos (SSH 22022, MySQL 33306), firewall UFW con política de denegación por defecto en ambas VM, una base de datos que solo acepta conexiones del servidor web, un usuario de MySQL limitado a la base de WordPress y un único usuario (`wpadmin`) con escritura sobre los archivos del sitio, al que se accede por SFTP enjaulado. El certificado se obtuvo con el reto DNS-01, el único de los métodos de Let's Encrypt compatible con un servidor que no es accesible desde Internet.

Toda la configuración está en scripts versionados que leen un archivo común de parámetros, lo que permitió validar el procedimiento en un prototipo y repetirlo sin cambios en la laptop de la demostración. Las pruebas cubren cada requisito del enunciado y sus resultados se presentan en la sección 8.

Tabla 1. *Cumplimiento de requisitos.*

Requisito del enunciado	Resultado	Evidencia
2 VM con Ubuntu Server 26.04 LTS sin GUI	Cumplido · 26.04.1 LTS	Fig. 4, 21
Capa web con cliente MySQL	Cumplido · Nginx + PHP-FPM, mysql 8.4.11	Fig. 9, 20
BD en VM aparte, usuario propio de WordPress	Cumplido · wp_user, sin privilegios globales	Fig. 7, 8
HTTPS con certificado de Let's Encrypt	Cumplido · válido hasta 07/02/2027	Fig. 17-19
SFTP solo para el administrador designado	Cumplido · wpadmin, chroot /var/www	Fig. 11-15
SSH y MySQL en puertos no conocidos	Cumplido · 22022 y 33306	Fig. 5, 7
UFW habilitado solo con puertos usados	Cumplido · ambas VM	Fig. 6
Administración remota (incluida la BD)	Cumplido · solo SSH	Fig. 8, 21
Acceso por nombre de dominio local	Cumplido · archivo hosts	Fig. 4

Índice

Resumen 2
1. Introducción y objetivos 4
1.1 Alcance 4
2. Organización del trabajo 4
3. Hipervisor y versiones de software 6
4. Arquitectura y direccionamiento 8
4.1 Decisiones de diseño 9
5. Procedimiento por etapas 10
5.1 Máquinas virtuales y red 10
5.2 Acceso SSH y puerto no conocido 11
5.3 Firewall UFW 12
5.4 Capa de base de datos 12
5.5 Capa web 13
5.6 Permisos con chown y chmod 14
5.7 SFTP y FileZilla 14
5.8 HTTPS con Let's Encrypt 16
5.9 Resolución de nombres local 19
6. Cliente y servidor 20
7. Seguridad: defensa en profundidad 20
8. Pruebas de funcionamiento 22
9. Problemas encontrados y soluciones 24
10. Conclusiones 24
11. Referencias 25
Anexo A. Repositorio y scripts 26

1. Introducción y objetivos

El laboratorio plantea montar un servicio web real con los componentes que se encuentran en un centro de datos pequeño: servidores virtualizados, una red interna, separación entre aplicación y datos, acceso administrativo remoto y publicación cifrada. El enunciado no entrega comandos; cada herramienta y versión se eligió y se justifica en este documento.

La tabla siguiente relaciona los objetivos de la actividad con las secciones donde se desarrollan y con la evidencia que los demuestra.

Tabla 2. *Objetivos de la actividad y su desarrollo.*

Objetivo	Cómo se aborda	Sección
Permisos de archivos (chown/chmod)	Modelo propietario/grupo/otros que deja la escritura solo a wpadmin y la lectura a www-data	5.6
Gestión remota por SSH	Ambas VM y MySQL se administran solo por SSH con llave; la consola del hipervisor no se usa	5.2, 5.4
Cliente y servidor	Cada conexión se analiza como aplicación y como ordenador; la VM web es cliente de la BD	6
Resolución de nombres local	El archivo hosts del host tiene prioridad sobre el DNS y dirige el dominio a la VM	5.9
Servicio de 2 capas seguro	Capas separadas, firewall, puertos no conocidos, TLS y mínimo privilegio	4, 7

1.1 Alcance

El documento describe la implementación presentada en clase (laptop con Windows 11 y VirtualBox). Los scripts se probaron antes en un prototipo y después se ejecutaron sin cambios en la máquina de la demostración.

2. Organización del trabajo

El trabajo se dividió por capas para que cada integrante fuera responsable de un conjunto de requisitos completo y pudiera defenderlo. El repositorio fue el punto de integración: todos los scripts leen los mismos parámetros de `config/lab5.env` y las credenciales de `secretos.env`, que no se versiona.

Tabla 3. *Distribución de responsabilidades.*

Integrante	Responsabilidades
Daniela Ramírez	Creación de las VM en VirtualBox con instalación desatendida (autoinstall), red host-only y NAT, llaves SSH, archivo hosts, FileZilla, ejecución en la laptop de la demostración y registro de evidencias.
Fernando Mendoza	SSH en puerto no conocido, firewall UFW en ambas VM, capa de base de datos MySQL, pruebas de conectividad (host y servidores).

Integrante	Responsabilidades
Nicolás Concuá	Estructura del repositorio y configuración compartida, prototipo, capa web (Nginx, PHP-FPM, WordPress), permisos, SFTP y HTTPS con Let's Encrypt.

El orden de ejecución sigue las dependencias técnicas: primero la infraestructura, luego el acceso y el firewall, después la base de datos (la capa web la necesita para instalar WordPress), la capa web y por último las pruebas.

3. Hipervisor y versiones de software

Se usó **VirtualBox 7.2.0**, un hipervisor de tipo 2: se ejecuta como aplicación sobre el sistema operativo del host y comparte su kernel para el acceso al hardware. Es uno de los dos permitidos por el enunciado, es gratuito y ofrece por VM los dos tipos de red que necesita el diseño (NAT y host-only). Para dos servidores de 2 GB de RAM sin interfaz gráfica, la sobrecarga de un hipervisor de tipo 2 no es relevante.

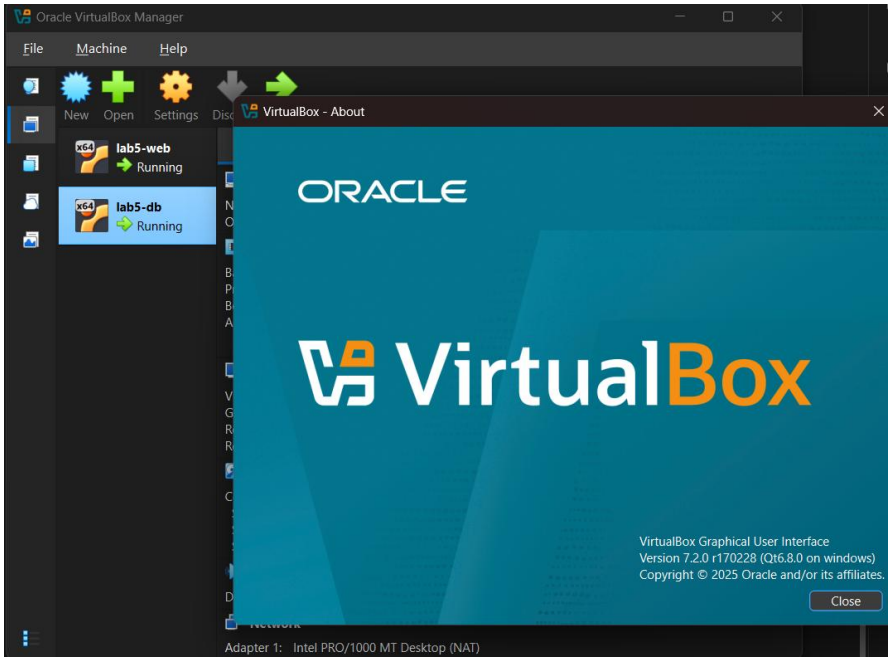


Figura 1. *VirtualBox 7.2.0 con las VM lab5-web y lab5-db en ejecución (demo).*

Las versiones de los servicios se tomaron de los repositorios oficiales de Ubuntu 26.04. Esto asegura que reciban actualizaciones de seguridad durante todo el soporte del sistema sin agregar repositorios de terceros, y que las versiones sean las que se probaron en el prototipo.

Tabla 4. *Versiones utilizadas y criterio de selección.*

Componente	Versión	Justificación
Hipervisor	VirtualBox 7.2.0 r170228	Permitido por el enunciado; redes NAT y host-only por VM; sin costo.
Sistema operativo	Ubuntu Server 26.04.1 LTS (kernel 7.0)	Requisito; LTS con 5 años de soporte; instalación mínima sin GUI.
SSH / SFTP	OpenSSH 10.2p1	Incluido en Ubuntu; soporta chroot y ForceCommand por usuario.
Firewall	UFW 0.36.2	Requisito; interfaz simple sobre nftables, reglas por puerto y origen.
Servidor web	Nginx 1.28.3	Menor uso de memoria que Apache; configuración TLS concisa.
Intérprete PHP	PHP-FPM 8.5.4	PHP en un proceso separado de Nginx, con su propio usuario.

Componente	Versión	Justificación
CMS	WordPress 7.1.4 · WP-CLI 2.12.0	Versión estable vigente; WP-CLI permite instalar sin asistente web.
Base de datos	MySQL 8.4.11 LTS	Requisito (MySQL); rama LTS del repositorio de Ubuntu 26.04.
Certificados	certbot 4.0.0	Cliente ACME de la EFF; hooks para el reto DNS-01.
Cliente SFTP	FileZilla Client	Requisito del enunciado; SFTP sobre SSH.

4. Arquitectura y direccionamiento

La arquitectura separa la presentación (capa web) de los datos (capa de base de datos) en dos VM distintas. Cada VM tiene dos adaptadores: el adaptador 1 en NAT, solo para salir a Internet (paquetes, Let's Encrypt, API de DuckDNS), y el adaptador 2 en una red host-only con IP fija, por donde se ofrecen todos los servicios. Así ningún servicio queda expuesto a la red física y el host se comporta como el único cliente del laboratorio.

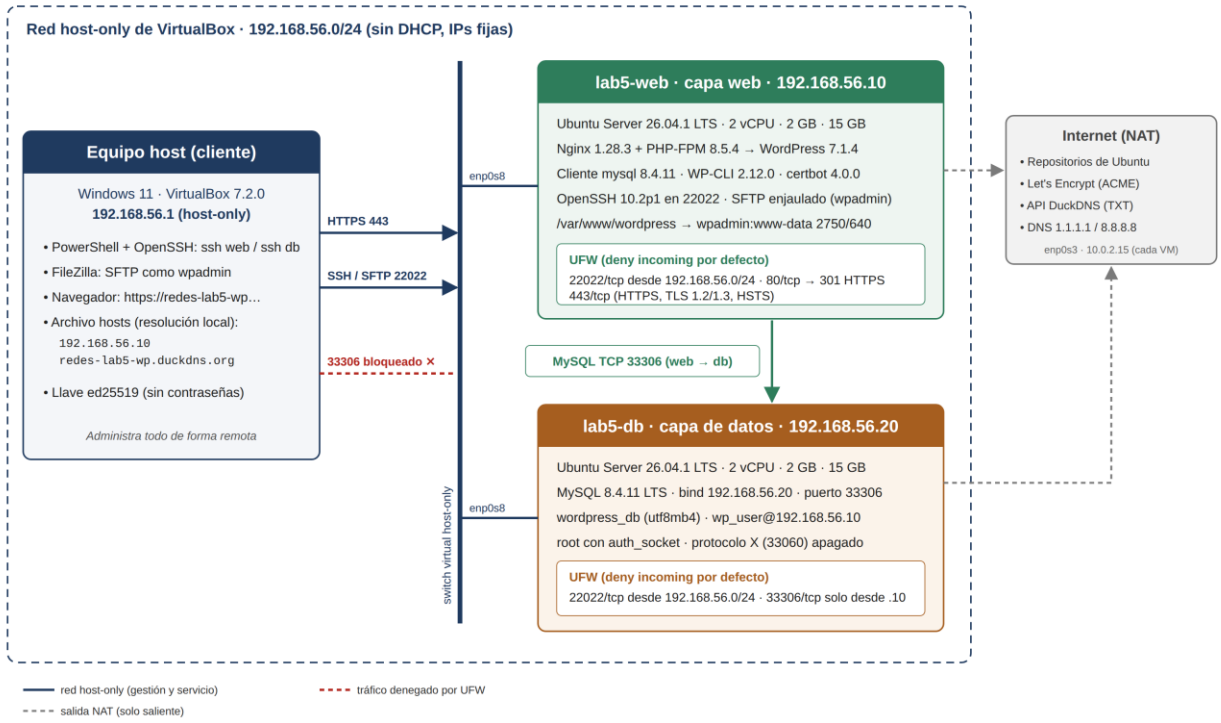


Figura 2. *Diagrama de la implementación: equipos, direcciones, servicios, puertos y reglas de firewall.*

Tabla 5. *Direccionamiento y puertos.*

Equipo	Nombre / IP	Servicios expuestos (puerto por defecto → usado)
Host (cliente)	Windows 11 · 192.168.56.1	Ninguno; usa clientes SSH, SFTP y navegador
VM capa web	lab5-web · 192.168.56.10/24	SSH/SFTP 22 → 22022; HTTPS 443; HTTP 80 (solo redirige)
VM capa de datos	lab5-db · 192.168.56.20/24	SSH 22 → 22022; MySQL 3306 → 33306; protocolo X 33060 desactivado
Dominio	redes-lab5-wp.duckdns.org	Resuelto a 192.168.56.10 por el archivo hosts del host

4.1 Decisiones de diseño

Tabla 6. *Decisiones de diseño y alternativas.*

Decisión	Alternativa descartada	Motivo
IPs fijas sin DHCP en host-only	DHCP de VirtualBox	Firewall, bind-address de MySQL, usuario wp_user@IP y archivo hosts dependen de direcciones estables.
Dos adaptadores por VM (NAT + host-only)	Adaptador puente	El puente expone las VM a la red física; NAT solo permite tráfico saliente.
Puertos 22022 y 33306	22 y 3306	Reduce escaneos y ataques automatizados; complementa a llaves y firewall.
Nginx + PHP-FPM	Apache + mod_php	Menor memoria en VM de 2 GB; PHP aislado en su propio proceso y usuario.
Reto DNS-01 con DuckDNS	HTTP-01, TLS-ALPN-01, certificado autofirmado	Los dos primeros requieren que Let's Encrypt llegue a la VM; el autofirmado no se acepta.
Scripts versionados	Configuración manual	Procedimiento reproducible y revisable; mismo resultado en prototipo y demo.

5. Procedimiento por etapas

Cada etapa se ejecutó desde el host con un comando de la forma `ssh -t web "cd ~/lab5 && sudo bash scripts/<etapa>.sh"`. Para cada una se indica el objetivo, la decisión tomada, lo que hace el script y cómo se verificó. Las figuras con salida de terminal reproducen el texto registrado durante la ejecución en la laptop de la demostración (carpeta `evidencias/` del repositorio).

Tabla 7. *Orden de ejecución.*

#	Etapa	Script	Dónde
1	Creación de VM y red	infra-windows/crear-vms.sh	Host
2	SSH en 22022	scripts/comun/01-ssh-puerto.sh	web y db
3	Firewall UFW	scripts/comun/02-ufw.sh	web y db
4	MySQL	scripts/bd/01-mysql.sh	db
5	Nginx, PHP, cliente MySQL, WP-CLI	scripts/web/01-stack-web.sh	web
6	WordPress y permisos	scripts/web/02-wordpress-permisos.sh	web
7	SFTP para wpadmin	scripts/web/03-sftp.sh	web
8	HTTPS con Let's Encrypt	scripts/web/04-https.sh	web
9	Archivo hosts y FileZilla	Manual	Host
10	Pruebas	scripts/pruebas/*	Host y VM

5.1 Máquinas virtuales y red

Objetivo. Disponer de dos servidores Ubuntu Server 26.04 sin entorno gráfico, conectados entre sí y con el host por una red privada, y con salida a Internet.

Implementación. `crear-vms.sh` crea las VM (2 vCPU, 2 GB de RAM, disco de 15 GB) con `VBoxManage` y las instala de forma desatendida con *autoinstall*: una ISO auxiliar con `user-data` define hostname, usuario `sysadmin`, llave pública ed25519 del host, zona horaria y la configuración de red de netplan. Las interfaces se identifican por MAC: `enp0s3` (NAT, DHCP, DNS 1.1.1.1 y 8.8.8.8) y `enp0s8` (host-only, IP fija). La red host-only se configuró sin servidor DHCP. La consola del hipervisor solo se usó durante la instalación; a partir de ahí todo se hizo por SSH.

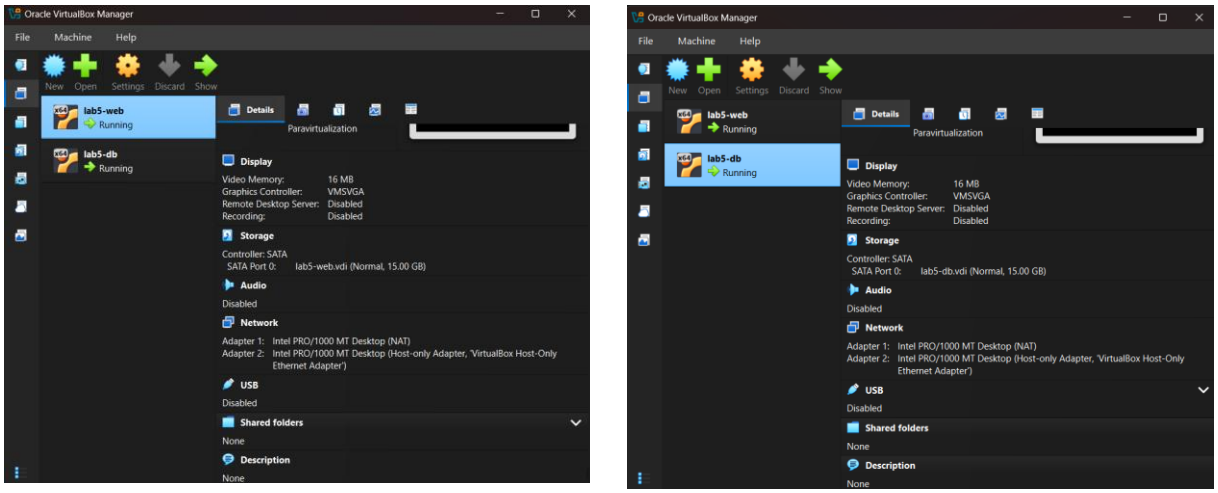


Figura 3. *Adaptadores de lab5-web (izquierda) y lab5-db (derecha): NAT y host-only.*

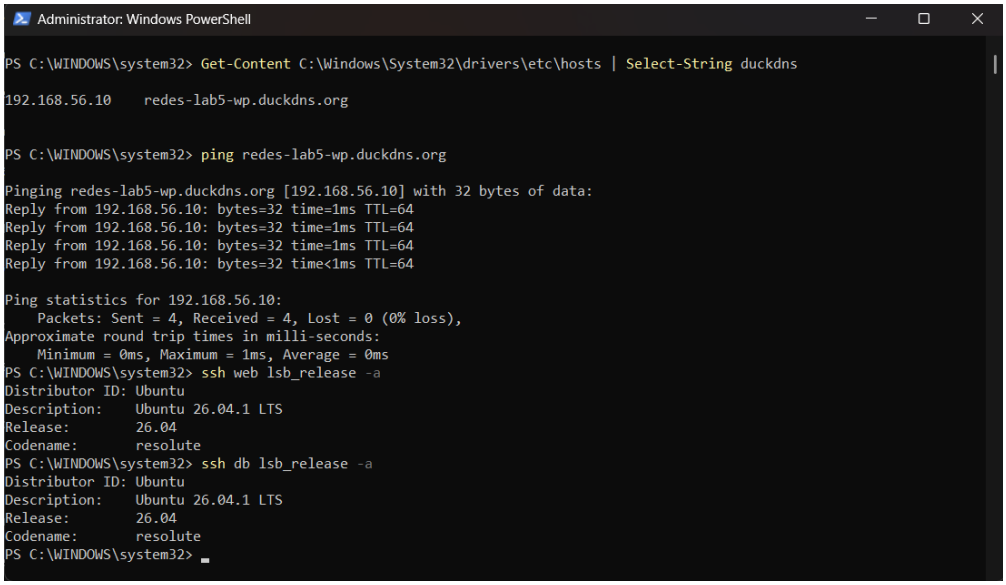


Figura 4. *Resolución local del dominio, ping a la VM web y versión del sistema en ambas VM por SSH.*

5.2 Acceso SSH y puerto no conocido

Objetivo. Administrar ambas VM sin acceso físico y reducir la superficie de ataque del servicio SSH.

Decisión. Autenticación solo con llave ed25519 y puerto 22022. La llave elimina los ataques de fuerza bruta contra contraseñas; el puerto no conocido reduce el ruido de escáneres automatizados que prueban el 22. Este último es una medida de oscuridad: complementa a la llave y al firewall, no los reemplaza.

Implementación. `01-ssh-puerto.sh` crea `/etc/ssh/sshd_config.d/10-lab5.conf` con `Port 22022`, `PermitRootLogin no`, `PasswordAuthentication no`, `KbdInteractiveAuthentication no` y `X11Forwarding no`. Usar un archivo en `sshd_config.d` en lugar de editar `sshd_config` aísla la configuración propia de las actualizaciones del paquete. Antes de aplicar el cambio, `sshd -t` valida la sintaxis para no perder el acceso remoto. En Ubuntu 24.04 y posteriores el puerto lo abre systemd (`ssh.socket`), por lo que el script ejecuta `systemctl daemon-reload` y reinicia el socket; reiniciar solo el servicio no cambia el puerto.

```
$ ssh -t web "cd ~/lab5 && sudo bash scripts/comun/01-ssh-puerto.sh"
LISTEN 0  4096       0.0.0.0:22022    0.0.0.0:*
users:(("sshd",pid=13511,fd=3),("systemd",pid=1,fd=82))
LISTEN 0  4096          [::]:22022       [::]:*
users:(("sshd",pid=13511,fd=4),("systemd",pid=1,fd=87))
OK: SSH escuchando en 22022
$ ssh -t db "cd ~/lab5 && sudo bash scripts/comun/01-ssh-puerto.sh"
LISTEN 0  4096       0.0.0.0:22022    0.0.0.0:*
users:(("sshd",pid=1587,fd=3),("systemd",pid=1,fd=73))
OK: SSH escuchando en 22022
PS> Test-NetConnection 192.168.56.10 -Port 22
RemotePort       : 22
TcpTestSucceeded : False
```

Figura 5. *SSH escuchando en 22022 en ambas VM; el puerto 22 ya no responde desde el host.*

Después del cambio, los alias `web` y `db` del archivo `~/.ssh/config` del host se actualizaron a `Port 22022`, de modo que la administración siguió siendo `ssh web` / `ssh db`.

5.3 Firewall UFW

Objetivo. Habilitar el firewall en ambos servidores y permitir únicamente los puertos que se usan.

Decisión. Política por defecto de denegar todo el tráfico entrante y permitir el saliente; cada regla se define por servicio y, cuando es posible, por origen. SSH solo se acepta desde la red del laboratorio. El puerto 80 queda abierto únicamente para responder con una redirección a HTTPS. En la VM de datos, el 33306 solo admite al servidor web (192.168.56.10): ni siquiera el host puede conectarse a MySQL. Esta regla y el `bind-address` de MySQL se cubren mutuamente; si una fallara, la base seguiría sin quedar expuesta.

```
$ ssh web "sudo ufw status verbose"
Status: active
Default: deny (incoming), allow (outgoing), disabled (routed)
To                         Action      From
22022/tcp                  ALLOW IN    192.168.56.0/24           # SSH/SFTP lab5
80/tcp                     ALLOW IN    Anywhere                  # HTTP (redirige a HTTPS)
443/tcp                    ALLOW IN    Anywhere                  # HTTPS WordPress
80/tcp (v6)                ALLOW IN    Anywhere (v6)             # HTTP (redirige a HTTPS)
443/tcp (v6)               ALLOW IN    Anywhere (v6)             # HTTPS WordPress

$ ssh db "sudo ufw status verbose"
Status: active
Default: deny (incoming), allow (outgoing), disabled (routed)
To                         Action      From
22022/tcp                  ALLOW IN    192.168.56.0/24           # SSH/SFTP lab5
33306/tcp                  ALLOW IN    192.168.56.10             # MySQL desde web
```

Figura 6. *Reglas de UFW en ambas VM (demo).*

5.4 Capa de base de datos

Objetivo. Ejecutar MySQL en una VM independiente con una base y un usuario propios de WordPress que no sean el administrador del motor, y gestionarla por SSH.

Decisión. MySQL 8.4 LTS, la rama de soporte largo que distribuye Ubuntu 26.04. El usuario `wp_user` solo puede conectarse desde 192.168.56.10 y solo tiene privilegios sobre `wordpress_db`. El usuario `root` de MySQL conserva la autenticación `auth_socket`: solo entra quien ya es root en la VM (`sudo mysql`), no tiene contraseña que pueda filtrarse y no acepta conexiones remotas.

Implementación. `01-mysql.sh` instala `mysql-server` y crea `/etc/mysql/mysql.conf.d/zz-lab5.cnf` con `bind-address = 192.168.56.20`, `port = 33306` y `mysqlx = 0` (apaga el protocolo X en 33060, que no se usa). El prefijo `zz-` es necesario porque MySQL lee esos archivos en orden alfabético y conserva el último valor de cada opción; con otro nombre, `mysqld.cnf` lo sobrescribía (sección 9). Luego crea la base en `utf8mb4` y el usuario con `CREATE USER IF NOT EXISTS` + `ALTER USER`, lo que hace el script idempotente.

```
$ ssh -t db "cd ~/lab5 && sudo bash scripts/bd/01-mysql.sh"
Grants for wp_user@192.168.56.10
GRANT USAGE ON *.* TO `wp_user`@`192.168.56.10`
GRANT ALL PRIVILEGES ON `wordpress_db`.* TO `wp_user`@`192.168.56.10`
mysql  Ver 8.4.11-0ubuntu0.26.04.1 for Linux on x86_64 ((Ubuntu))
LISTEN 0      151    192.168.56.20:33306      0.0.0.0:*    users:(("mysqld",pid=2609,fd=19))
```

Figura 7. *MySQL 8.4.11 escuchando solo en 192.168.56.20:33306 y privilegios de wp_user.*

```
$ ssh -t db "sudo mysql -e 'SELECT user, host, plugin FROM mysql.user;'"
user              host            plugin
wp_user           192.168.56.10   caching_sha2_password
debian-sys-maint  localhost       auth_socket
mysql.infoschema  localhost       caching_sha2_password
mysql.session     localhost       caching_sha2_password
mysql.sys         localhost       caching_sha2_password
root              localhost       auth_socket
```

Figura 8. *Gestión de la base por SSH: usuarios y método de autenticación.*

5.5 Capa web

Objetivo. Servir WordPress desde una VM con soporte PHP y con un cliente MySQL instalado, conectada a la base remota.

Implementación. `01-stack-web.sh` instala Nginx, PHP-FPM con las extensiones que requiere WordPress (`mysql`, `curl`, `gd`, `mbstring`, `xml`, `zip`, `intl`, `imagick`), el cliente `mysql` y WP-CLI; elimina el sitio por defecto y oculta la versión de Nginx (`server_tokens off`). `02-wordpress-permisos.sh` prueba primero la conexión remota con `wp_user` y después descarga, configura e instala WordPress con WP-CLI apuntando a `192.168.56.20:33306`. En `wp-config.php` se fijan `DISALLOW_FILE_EDIT`, `DISALLOW_FILE_MODS` y `FORCE_SSL_ADMIN`: el panel no intenta escribir en disco (no podría, ver 5.6) y el inicio de sesión siempre usa HTTPS.

```
$ ssh -t web "cd ~/lab5 && sudo bash scripts/web/02-wordpress-permisos.sh"
estado
OK conexión a BD remota
Downloading WordPress 7.1.4 (en_US)...
md5 hash verified: e3ab64c0de73fe22c0e482c00fa9298c
Success: Generated 'wp-config.php' file.
Success: Added the constant 'DISALLOW_FILE_MODS' to the 'wp-config.php' file with the raw value
'true'.
Success: Added the constant 'FORCE_SSL_ADMIN' to the 'wp-config.php' file with the raw value 'true'.
Success: WordPress installed successfully.
Success: Language activated.
```

Figura 9. *Instalación de WordPress contra la base remota.*

5.6 Permisos con chown y chmod

En Linux cada archivo tiene un propietario, un grupo y permisos para tres niveles: propietario, grupo y otros. Cada nivel combina lectura (4), escritura (2) y ejecución (1); en un directorio, la ejecución significa poder entrar en él. `chown` cambia propietario y grupo, y `chmod` cambia los permisos.

Diseño. Se creó `wpadmin`, sin shell (`/usr/sbin/nologin`), como único usuario con escritura sobre el sitio. El grupo de los archivos es `www-data`, el usuario con el que corren Nginx y PHP-FPM, de modo que el servicio puede leer pero no modificar. Los demás usuarios, incluido `sysadmin`, no tienen ningún permiso. Si un atacante explotara una vulnerabilidad de WordPress, su código correría como `www-data` y no podría alterar archivos ni dejar código persistente; además, `wp-config.php`, que contiene la contraseña de la base, no es legible por otros usuarios.

Tabla 8. *Modelo de permisos del sitio.*

Ruta	Propietario:grupo	Modo	Significado
/var/www	root:root	755	Lo exige el chroot de SFTP: la jaula debe pertenecer a root
/var/www/wordpress y subdirectorios	wpadmin:www-data	2750	rwx para wpadmin, r-x para www-data, nada para otros; setgid (2): lo nuevo hereda el grupo www-data
Archivos	wpadmin:www-data	640	rw- para wpadmin, r-- para www-data, nada para otros

```
chown -R wpadmin:www-data /var/www/wordpress
find /var/www/wordpress -type d -exec chmod 2750 {} +
find /var/www/wordpress -type f -exec chmod 640 {} +
```

Figura 10. *Comandos aplicados por 02-wordpress-permisos.sh.*

```
$ ssh -t web "cd ~/lab5 && sudo bash scripts/pruebas/pruebas-servidor.sh web"
drwxr-xr-x 3 root    root     4096 Nov  9 14:01 /var/www
drwxr-s--- 5 wpadmin www-data 4096 Nov  9 14:01 /var/www/wordpress
-rw-r-----  1 wpadmin www-data   405 Nov  9 14:01 index.php
-rw-r-----  1 wpadmin www-data 19903 Nov  9 14:01 license.txt
===== www-data NO puede escribir =====
touch: cannot touch '/var/www/wordpress/prueba': Permission denied
===== sysadmin NO puede escribir =====
touch: setting times of '/var/www/wordpress/prueba': Permission denied
===== wpadmin SÍ puede escribir =====
-rw-r--r-- 1 wpadmin www-data 0 Nov  9 14:04 /var/www/wordpress/prueba
```

Figura 11. *Permisos resultantes y prueba de escritura con los tres usuarios (demo).*

5.7 SFTP y FileZilla

Objetivo. Permitir la escritura sobre la carpeta de WordPress por SFTP únicamente al administrador designado.

Decisión. SFTP viaja cifrado dentro de SSH, por lo que no hace falta un servidor FTP adicional ni abrir más puertos. `03-sftp.sh` agrega un bloque `Match User wpadmin` en `/etc/ssh/sshd_config.d/20-sftp-wpadmin.conf` con `ChrootDirectory /var/www` (wpadmin solo ve `/wordpress`), `ForceCommand internal-sftp -u 0027` (no puede abrir una terminal y lo que sube queda 640/750), contraseña

habilitada solo para él (para FileZilla) y túneles deshabilitados. `sysadmin` sigue entrando únicamente con llave.

```
== Config efectiva para wpadmin          == Config efectiva para sysadmin
port 22022                               port 22022
passwordauthentication yes               passwordauthentication no
forcecommand internal-sftp -u 0027       forcecommand none
chrootdirectory /var/www                 chrootdirectory none
```

Figura 12. *Configuración efectiva de sshd para cada usuario (sshd -T).*

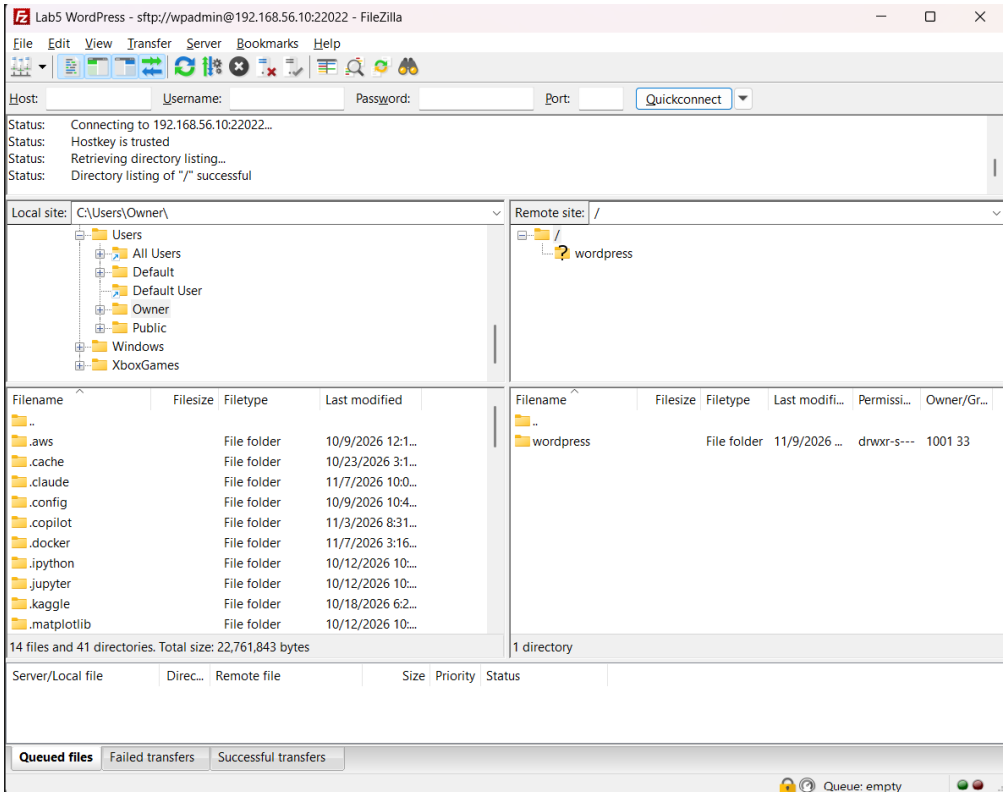


Figura 13. *FileZilla conectado como wpadmin a 192.168.56.10:22022: la jaula solo muestra /wordpress.*

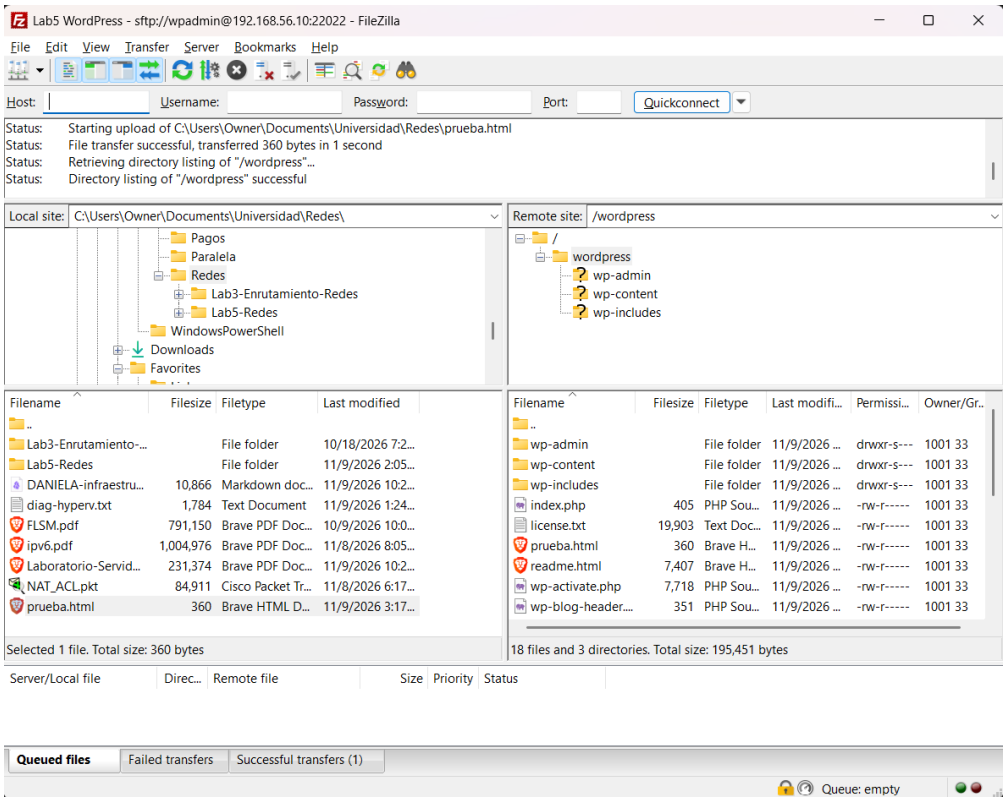


Figura 14. *Subida de prueba.html por SFTP: el archivo queda como -rw-r----- wpadmin www-data (1001:33).*



Figura 15. *El archivo subido se sirve por HTTPS: el servidor web lo lee gracias al grupo www-data.*

5.8 HTTPS con Let's Encrypt

Let's Encrypt solo emite certificados a quien demuestra control sobre el dominio. Ofrece tres métodos de validación:

Tabla 9. *Métodos de validación de Let's Encrypt.*

Método	Cómo valida	¿Aplica a este diseño?
HTTP-01	Descarga un archivo desde el puerto 80 del servidor	No: la VM no es accesible desde Internet
TLS-ALPN-01	Negocia TLS con el servidor en el puerto 443	No: mismo motivo
DNS-01	Consulta un registro TXT en el DNS público del dominio	Sí: no necesita conectarse al servidor

Decisión. Se eligió DNS-01 con el subdominio gratuito `redes-lab5-wp.duckdns.org`, porque DuckDNS ofrece una API para publicar el registro TXT. Un dominio inventado (`.local`, `.lab`) no existe en el DNS público y no puede validarse. La elección es compatible con la resolución local: el nombre existe públicamente solo para la validación, mientras que el archivo hosts del host lo dirige a la IP privada. El navegador acepta el certificado porque verifica el nombre y la cadena de confianza, no la dirección IP a la que resolvió.

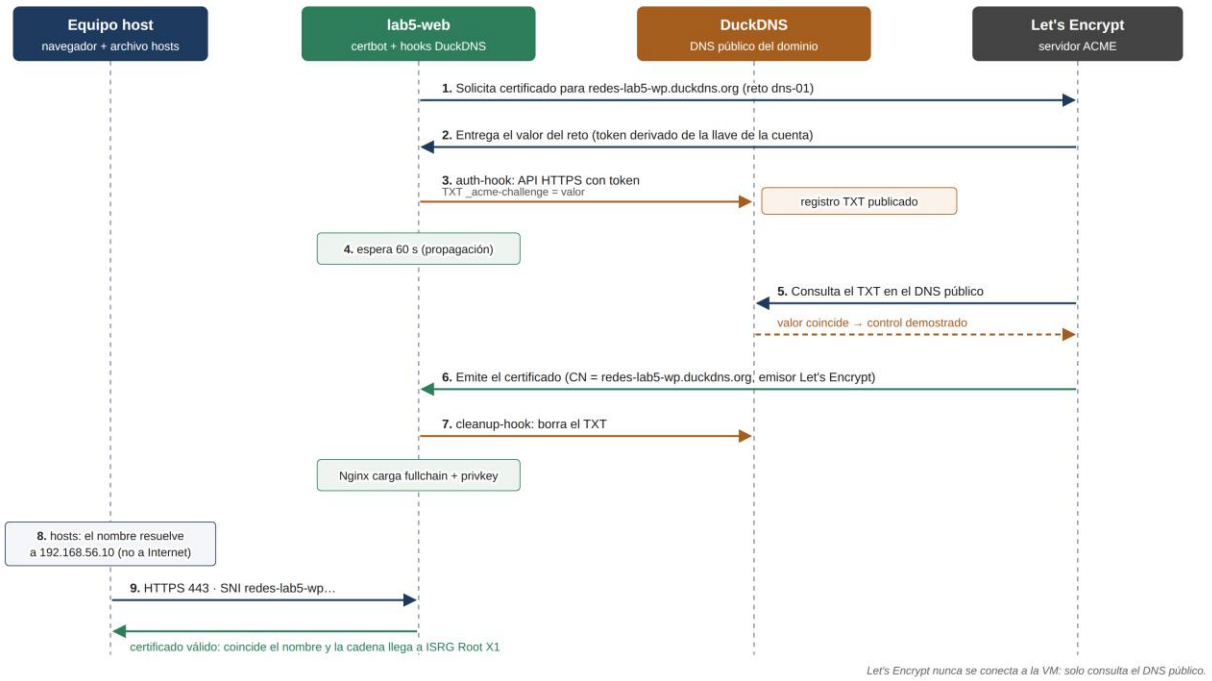


Figura 16. *Secuencia de emisión con DNS-01 y uso del certificado desde el host.*

Implementación. `04-https.sh` instala certbot y dos *hooks*: el primero publica el valor del reto en DuckDNS y espera 60 s a que se propague; el segundo borra el registro al terminar. El token de DuckDNS se guarda en `/etc/letsencrypt/duckdns/credenciales` con permisos 600. Primero se ejecutó contra el entorno de pruebas (*staging*) para no consumir el límite de 5 certificados idénticos por semana. Nginx quedó con redirección 301 de HTTP a HTTPS, TLS 1.2 y 1.3, HTTP/2, HSTS y bloqueo de `wp-config.php`, `xmlrpc.php` y archivos ocultos. `certbot.timer` renueva el certificado automáticamente con los mismos hooks.

```
$ ssh -t web "cd ~/lab5 && sudo bash scripts/web/04-https.sh"
Requesting a certificate for redes-lab5-wp.duckdns.org
Hook '--manual-auth-hook' for redes-lab5-wp.duckdns.org ran with output:
 DuckDNS: OK
Successfully received certificate.
Certificate is saved at: /etc/letsencrypt/live/redes-lab5-wp.duckdns.org/fullchain.pem
This certificate expires on 2027-02-07.
nginx: configuration file /etc/nginx/nginx.conf test is successful
issuer=C=US, O=Let's Encrypt, CN=YE2
subject=CN=redes-lab5-wp.duckdns.org
```

Figura 17. *Emisión del certificado en la VM web (demo).*

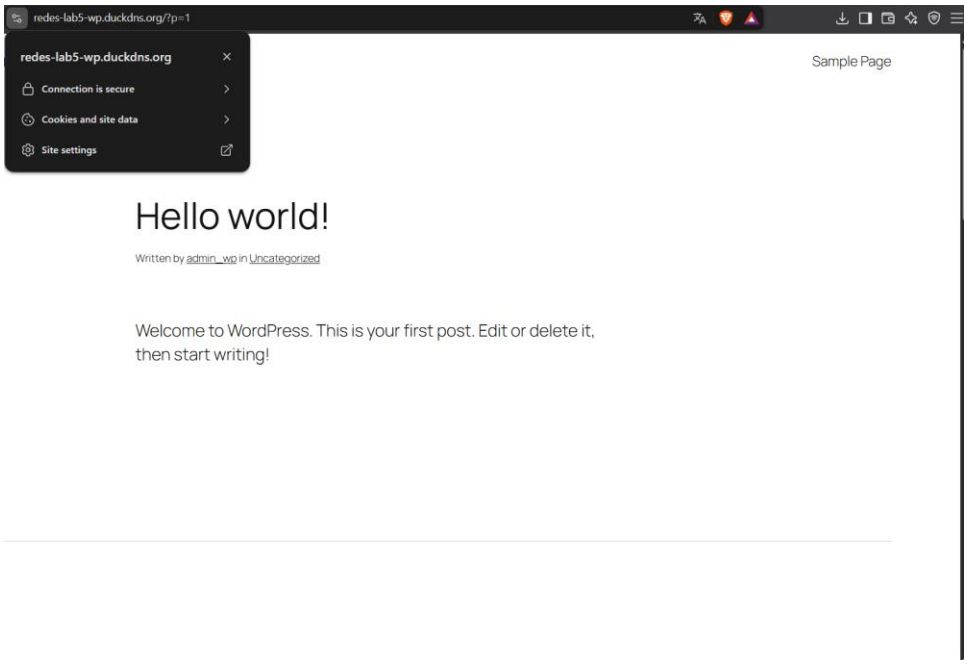


Figura 18. *El navegador del host indica conexión segura con el dominio.*

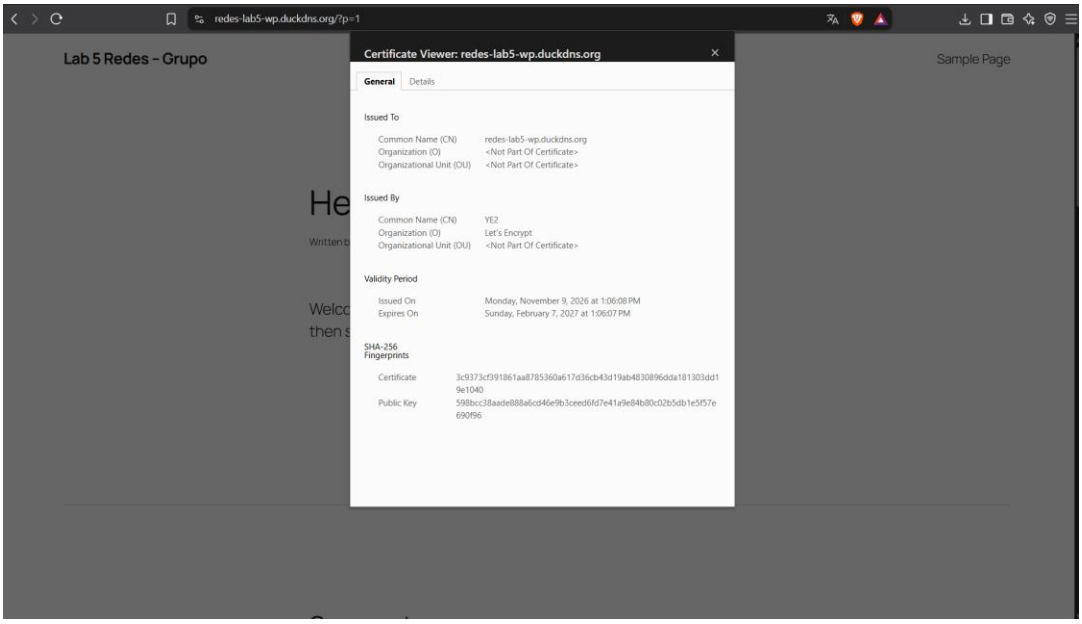


Figura 19. *Certificado emitido por Let's Encrypt (YE2) para redes-lab5-wp.duckdns.org, válido hasta el 07/02/2027.*

5.9 Resolución de nombres local

Se agregó la línea `192.168.56.10 redes-lab5-wp.duckdns.org` al archivo `C:\Windows\System32\drivers\etc\hosts` y se vació la caché con `ipconfig /flushdns`. El resolver del sistema operativo consulta ese archivo antes que a los servidores DNS configurados. Sin la línea, el nombre resolvía a la IP pública registrada en DuckDNS; con ella, resuelve a la VM web (Figura 4). Es el mismo mecanismo que usa `/etc/hosts` en Linux y macOS.

6. Cliente y servidor

Los términos cliente y servidor se usan en dos sentidos. Como **aplicación**, el servidor es el programa que ofrece un servicio y el cliente el que lo consume. Como **ordenador**, son las máquinas que cumplen esos papeles. Una misma máquina puede ser servidor en una conexión y cliente en otra: `lab5-web` es servidor para el navegador del host y, al mismo tiempo, cliente de la base de datos.

Tabla 10. *Roles de cliente y servidor en cada conexión.*

Conexión	Aplicación cliente → servidor	Ordenador cliente → servidor	Protocolo / puerto
Sitio web	Navegador → Nginx + PHP-FPM	Host → lab5-web	HTTPS 443
Archivos del sitio	FileZilla → OpenSSH (internal-sftp)	Host → lab5-web	SFTP 22022
Administración	ssh → sshd	Host → lab5-web y lab5-db	SSH 22022
Datos de WordPress	PHP (mysqli) y cliente mysql → mysqld	lab5-web → lab5-db	MySQL 33306
Certificado	certbot → Let's Encrypt y API DuckDNS	lab5-web → servidores en Internet	HTTPS 443 (NAT)
Resolución local	Resolver del SO → archivo hosts	Host (consulta local)	—

7. Seguridad: defensa en profundidad

Ningún control se considera suficiente por sí solo. La tabla resume las capas aplicadas y el riesgo que mitiga cada una.

Tabla 11. *Controles de seguridad aplicados.*

Control	Ámbito	Riesgo que mitiga
Llave ed25519, sin contraseñas ni login de root por SSH	SSH	Fuerza bruta y robo de contraseñas
SSH en 22022 y MySQL en 33306	Red	Escaneos y ataques automatizados a puertos conocidos
UFW con denegación por defecto y reglas por origen	Red	Exposición de servicios no previstos
bind-address en la IP host-only + regla 33306 solo desde .10	Base de datos	Acceso directo a MySQL desde otros equipos
wp_user limitado a wordpress_db; root con auth_socket	Base de datos	Escalamiento si se filtra wp-config.php
wpadmin único con escritura; www-data solo lectura; otros sin acceso	Archivos	Modificación del sitio o persistencia tras explotar WordPress
SFTP enjaulado con ForceCommand y sin túneles	SFTP	Navegación del sistema y uso como shell o proxy

Control	Ámbito	Riesgo que mitiga
DISALLOW_FILE_MODS y DISALLOW_FILE_EDIT	WordPress	Instalación de plugins o edición de código desde el panel
TLS 1.2/1.3, HSTS y redirección 80 → 443	Transporte	Intercepción y degradación a HTTP
Bloqueo de wp-config.php, xmlrpc.php y ocultos; server_tokens off	Nginx	Fuga de información y fuerza bruta por XML-RPC
secretos.env fuera del repositorio; token de DuckDNS con permisos 600	Operación	Filtración de credenciales

8. Pruebas de funcionamiento

Las pruebas están automatizadas en `scripts/pruebas/`: `pruebas-servidor.sh` se ejecuta dentro de cada VM y `pruebas-host-windows.ps1` desde el host. Cada prueba verifica un requisito y, cuando corresponde, también su caso negativo (que lo que debe estar cerrado efectivamente lo esté).

Tabla 12. *Matriz de pruebas.*

Requisito	Prueba	Resultado esperado	Resultado obtenido
Conectividad entre capas	ping web → db y db → web	0 % de pérdida	0 % (Fig. 20)
Puerto de MySQL	nc -zv 192.168.56.20 33306 desde web	Conecta	succeeded
Puerto por defecto cerrado	nc -zv 192.168.56.20 3306 desde web	Sin respuesta	timed out
2 capas: login remoto	mysql -h 192.168.56.20 -P 33306 -u wp_user	Usuario, host y tablas wp_*	wp_user@192.168.56.10, lab5-db, 33306, 12 tablas
MySQL no accesible desde el host	Test-NetConnection 192.168.56.20 -Port 33306	False	False (Fig. 21)
SSH en puerto no conocido	Test-NetConnection a 22022 y 22	22022 True, 22 False	Cumple
Gestión remota	ssh web / ssh db con lsb_release	Hostname y Ubuntu 26.04.1	Cumple (Fig. 21)
Permisos	touch como www-data, sysadmin y wpadmin	Solo wpadmin escribe	Cumple (Fig. 11)
SFTP	FileZilla sube un archivo y se abre en el sitio	Archivo visible por HTTPS	Cumple (Fig. 14, 15)
HTTPS válido	curl -I sin -k desde el host; visor del navegador	200 y emisor Let's Encrypt	HTTP 200, YE2 (Fig. 19)
Redirección HTTP	curl -I http://redes-lab5-wp.duckdns.org	301 a https://	Cumple
Resolución local	hosts + ping al dominio	192.168.56.10	Cumple (Fig. 4)

```
$ ssh -t web "cd ~/lab5 && sudo bash scripts/pruebas/pruebas-servidor.sh web"
===== ping a la BD =====
3 packets transmitted, 3 received, 0% packet loss, time 2007ms
===== Puerto MySQL abierto en la BD (nc) =====
Connection to 192.168.56.20 33306 port [tcp/*] succeeded!
===== Puerto 3306 por defecto CERRADO =====
nc: connect to 192.168.56.20 port 3306 (tcp) timed out: Operation now in progress
===== Login MySQL remoto con wp_user =====
CURRENT_USER()          @@hostname   @@port
wp_user@192.168.56.10   lab5-db      33306
Tables_in_wordpress_db: wp_commentmeta wp_comments wp_links wp_options wp_postmeta wp_posts
                        wp_term_relationships wp_term_taxonomy wp_termmeta wp_terms wp_usermeta
wp_users
===== HTTPS local =====
HTTP/2 200
===== Certificado =====
issuer=C=US, O=Let's Encrypt, CN=YE2
subject=CN=redes-lab5-wp.duckdns.org
```

Figura 20. *Pruebas dentro de la VM web: conectividad, puertos, login remoto y certificado (demo).*

```
PS> powershell -ExecutionPolicy Bypass -File scripts\pruebas\pruebas-host-windows.ps1 -Dominio redes-
lab5-wp.duckdns.org
===== Puertos abiertos =====
192.168.56.10  RemotePort 22022  TcpTestSucceeded : True
192.168.56.10  RemotePort 443    TcpTestSucceeded : True
192.168.56.20  RemotePort 22022  TcpTestSucceeded : True
===== Puertos que deben estar CERRADOS (22 y MySQL desde host) =====
192.168.56.10  RemotePort 22     TcpTestSucceeded : False
192.168.56.20  RemotePort 33306  TcpTestSucceeded : False
===== SSH a ambas VM =====
lab5-web   Description: Ubuntu 26.04.1 LTS
lab5-db    Description: Ubuntu 26.04.1 LTS
===== HTTPS con certificado valido =====
HTTP/1.1 200 OK
Server: nginx
```

Figura 21. *Pruebas desde el host Windows (demo).*

La prueba con telnet que menciona la rúbrica se realiza en vivo durante la presentación (guion en `informe/Guion_Demostracion.md`): desde el host, `telnet 192.168.56.10 443` conecta y `telnet 192.168.56.20 33306` no; desde la VM web, `telnet 192.168.56.20 33306` muestra el saludo de MySQL 8.4.11.

9. Problemas encontrados y soluciones

Tabla 13. *Problemas y soluciones.*

Problema	Causa	Solución
apt no resolvía nombres en las VM	VirtualBox entregaba los DNS de la red de la universidad, que fallaban de forma intermitente	DNS públicos fijos (1.1.1.1, 8.8.8.8) en netplan para la interfaz NAT
MySQL escuchaba en 127.0.0.1	MySQL lee mysql.conf.d en orden alfabético; 99-lab5.cnf se leía antes que mysqld.cnf, que lo sobrescribía	Renombrar el archivo a zz-lab5.cnf; verificar con my_print_defaults mysqld
El puerto de SSH no cambiaba	Desde Ubuntu 24.04 el puerto lo abre ssh.socket (activación por socket)	systemctl daemon-reload y reinicio de ssh.socket
wp core download --locale=es_ES fallaba	No había traducción publicada para WordPress 7.1.4 al momento del prototipo	Descargar en inglés y activar el idioma después
server_tokens off no se aplicaba	En Nginx 1.28 de Ubuntu la línea es server_tokens build; y no está comentada	Expresión regular más general en sed
VM congeladas en la laptop Windows	Hyper-V activo (WSL2, integridad de memoria, VBS); VirtualBox corría sobre Hyper-V en lugar de usar AMD-V	Desactivar WSL/Hyper-V, integridad de memoria y VBS
El instalador pedía confirmar autoinstall	Sin el parámetro autoinstall en el kernel, Ubuntu pide confirmación	Responder con VBoxManage controlvm … keyboardputstring sin abrir la consola
FileZilla: Authentication failed	Contraseña pegada con un carácter adicional	Pegar solo el valor de la variable

10. Conclusiones

1. Separar la aplicación y los datos en VM distintas reduce la superficie de ataque solo si se acompaña de controles de red: con `bind-address`, la regla de UFW y un usuario limitado a su base, MySQL no es alcanzable ni siquiera desde el host, y una filtración de `wp-config.php` no compromete el motor.
2. El modelo de permisos propietario/grupo/otros permite separar quién administra el contenido de quién lo sirve. Con `wpadmin` como único escritor y `www-data` solo con lectura, una vulnerabilidad en WordPress no basta para modificar el sitio.
3. El reto DNS-01 resuelve el conflicto aparente entre un certificado público y un servidor privado: Let's Encrypt valida el control del dominio en el DNS público y el archivo hosts, que tiene prioridad sobre el DNS, lleva el nombre a la IP interna sin afectar la validez del certificado.
4. Administrar solo por SSH obliga a automatizar. Los scripts versionados hicieron posible repetir el mismo resultado en el prototipo y en la laptop de la demostración, y documentan con precisión lo que se configuró.
5. Los puertos no conocidos son útiles para reducir ruido, pero no son una medida de seguridad por sí mismos; la protección real está en las llaves, el firewall y el mínimo privilegio.

11. Referencias

[1] Canonical, "Ubuntu Server documentation: OpenSSH, Firewall (UFW) y Netplan", 2026. https://documentation.ubuntu.com/server/

[2] Oracle, "Oracle VirtualBox User Guide, cap. 6: Virtual Networking", versión 7.2, 2026. https://www.virtualbox.org/manual/

[3] Oracle, "MySQL 8.4 Reference Manual: Using Option Files; Server System Variables", 2026. https://dev.mysql.com/doc/refman/8.4/en/

[4] OpenBSD, "sshd_config(5): Match, ChrootDirectory, ForceCommand". https://man.openbsd.org/sshd_config

[5] F5/NGINX, "Module ngx_http_ssl_module" y "ngx_http_core_module". https://nginx.org/en/docs/

[6] Let's Encrypt, "Challenge Types" y "Rate Limits", 2026. https://letsencrypt.org/docs/

[7] Electronic Frontier Foundation, "Certbot User Guide: manual plugin and hooks". https://eff-certbot.readthedocs.io/

[8] DuckDNS, "Spec: update API and TXT records". https://www.duckdns.org/spec.jsp

[9] WordPress.org, "Hardening WordPress" y "WP-CLI Commands". https://developer.wordpress.org/

[10] Linux man-pages, chmod(1), chown(1), hosts(5), ufw(8).

[11] R. Barnes et al., "Automatic Certificate Management Environment (ACME)", RFC 8555, IETF, 2019.

Anexo A. Repositorio y scripts

Todo el procedimiento está en el repositorio del equipo. Los scripts son idempotentes (pueden ejecutarse de nuevo sin efectos adversos) y leen los parámetros de `config/lab5.env`.

Tabla 14. *Contenido del repositorio.*

Archivo	Función
config/lab5.env	IPs, puertos, usuarios y dominio comunes a todos los scripts (sin secretos)
secretos.env (no versionado)	Contraseñas y token de DuckDNS; plantilla en secretos.env.example
infra-windows/crear-vms.sh, subir.sh	Creación de las VM con autoinstall en VirtualBox y copia de scripts a ~/lab5
scripts/comun/01-ssh-puerto.sh	SSH en 22022, solo llave, sin root
scripts/comun/02-ufw.sh web|db	Firewall con denegación por defecto y reglas por rol
scripts/bd/01-mysql.sh	MySQL 8.4 en 192.168.56.20:33306, base y usuario de WordPress
scripts/web/01-stack-web.sh	Nginx, PHP-FPM, cliente MySQL y WP-CLI
scripts/web/02-wordpress-permisos.sh	wpadmin, WordPress con BD remota y permisos
scripts/web/03-sftp.sh	SFTP enjaulado para wpadmin
scripts/web/04-https.sh + hooks	Certificado DNS-01 con DuckDNS y Nginx con TLS
scripts/pruebas/*	Pruebas dentro de las VM y desde el host (Windows, macOS/Linux)
evidencias/	Salidas registradas y capturas de la demo